

TA-BRPL 연구 진행 연혁 복원본

Git 히스토리 기반 상세 기록

이건형

2026년 4월 2일

Abstract

이 문서는 TA-BRPL 저장소의 첫 커밋 `de6fbbe`부터 현재 스냅샷 `a55c3c1`까지의 Git 히스토리를 바탕으로, 연구가 실제로 어떤 순서로 전개되었는지 상세하게 복원한 연혁 문서다. 최종 알고리즘의 기술 요약이 아니라, 문제 정의, 초기 프로토타입, 외부 trust engine 실험, Contiki-NG/BRPL 통합, 고정 토폴로지 튜닝, 병렬 대량실험 인프라, 랜덤 토폴로지 일반화 검증, 논문화와 메인 endpoint 고정까지의 흐름을 날짜와 커밋 단위로 재구성한다. 이번 개정판은 여기에 더해 GitHub PR 본문과 리뷰 기록, 그리고 `contiki-ng-brpl` 서브모듈의 라우팅 코드 이력까지 함께 읽어, 특히 TA-BRPL 모델링이 어떤 실패와 수정 과정을 거쳐 정교화되었는지까지 복원한다. 커밋 메시지가 `..`, `fin`, `stable ver`처럼 설명적이지 않은 구간은 변경 파일과 동시기 문서를 함께 읽어 해석했으며, 해당 부분은 “Git 근거 기반 추론”으로 취급한다.

Contents

1	문서 목적과 복원 원칙	3
1.1	왜 이 문서를 따로 쓰는가	3
1.2	복원 자료	3
1.3	해석 원칙	3
2	전체 연구 흐름의 상위 요약	4
3	상세 연혁	4
3.1	2026-01-26: 최소 재현 가능한 실험 시스템 출발	4
3.2	2026-01-27: 공격자 도입과 phase 기반 실험 초안	5
3.3	2026-01-29 새벽: 랜덤 토폴로지와 내장 trust 개념의 등장	5
3.4	2026-01-29 저녁: 외부 trust engine과 로그 기반 피드백 루프 구축	6

3.5	2026-01-29 밤에서 2026-01-30 새벽: 모니터링, quick test, 첫 보고서 . . .	7
3.6	2026-02-06: Contiki-NG/BRPL 서브모듈 통합	7
3.7	2026-02-07: trust sweep 자동화와 안정화된 파이프라인	8
3.8	2026-02-08에서 2026-02-10: sinkhole 안정화와 보고서 누적	9
3.9	2026-02-11: 회귀 테스트 도입	9
3.10	2026-02-23에서 2026-03-04: 환경 재현성과 실험 운영성 정비	9
3.11	2026-03-05: 병렬 실험 인프라와 논문 초안의 시작	10
3.12	2026-03-16에서 2026-03-19: 워크스페이스 정리와 문서 버전업	11
3.13	2026-03-24 새벽: 통합 문서화와 메커니즘 재정리	11
3.14	2026-03-24 오전: 랜덤 토폴로지 프레임워크의 본격 도입	11
3.15	2026-03-27에서 2026-03-28: endpoint 고정과 실패의 체계적 문서화 . . .	12
3.16	2026-03-30: ablation 확장과 random-topology 보완 실험	13
4	TA-BRPL 모델링 진화의 상세 복원	14
4.1	초기 모델: seq-miss 기반 trust와 외부 주입형 구조	14
4.2	2026-02-06 전환: forwarding-ratio와 exposure 중심으로 재정의	14
4.3	초기 in-protocol trust adapter의 의미	15
4.4	2026-02-07 전후: lambda/gamma sweep과 파라미터화	15
4.5	2026-03 중반 이후: on-node 3축 trust 모델의 형성	16
4.6	trust_fwd 분리와 blackhole dilution 문제	16
4.7	validation layer의 등장: soft trust와 hard exclusion의 분리	17
4.8	BRPL 결합 함수의 진화: penalty에서 hysteresis-aware gating으로 . . .	17
4.9	정리: TA-BRPL 모델링 철학은 어떻게 바뀌었는가	18
5	요약 타임라인 표	18
6	복원으로부터 읽히는 연구 방향의 변화	20
6.1	1단계: “일단 돌려보자”	20
6.2	2단계: “trust를 어디에 넣어야 하는가”	20
6.3	3단계: “고정 토폴로지에서 왜 망하는가”	20
6.4	4단계: “일반화되는가”	20
6.5	5단계: “무엇을 성공이라고 부를 것인가”	20
7	최종 복원 결론	20

1 문서 목적과 복원 원칙

1.1 왜 이 문서를 따로 쓰는가

기존의 docs/base.md, docs/MAIN_EXPERIMENT_PROTOCOL.md, docs/TA_BRPL_V1_TO_NOW_POSTMORTEM은 각각 중요한 역할을 한다. 그러나 이 문서들은 주로 다음 중 하나에 집중한다.

- 현재 메커니즘 설명
- 실패 원인의 버전별 회고
- 메인실험 해석 기준의 사전 고정

반면 “처음 커밋부터 지금까지 실제로 연구가 어떤 순서로 확장되었는가”를 날짜 순으로 촘촘하게 복원한 문서는 별도로 없었다. 본 문서는 그 공백을 채운다.

1.2 복원 자료

복원에 사용한 근거는 다음과 같다.

- 전체 Git 커밋 로그와 날짜
- 각 전환점 커밋의 변경 파일 목록
- GitHub PR 본문과 리뷰 코멘트
- contiki-ng-brpl 서브모듈의 rpl-brpl.c 변경 이력
- 초기 readme.md와 docs/project_overview.md
- 중간 구조 문서와 실험 계획 문서
- 2026년 3월에 정리된 postmortem 및 main protocol 문서

1.3 해석 원칙

1. 커밋 메시지가 설명적일 경우 해당 메시지를 우선 신뢰한다.
2. 커밋 메시지가 불명확할 경우, 추가/수정된 파일의 축을 보고 해석한다.
3. 한 시점의 연구 의도를 추정할 때는 동시기 문서의 표현을 함께 본다.
4. GitHub PR과 리뷰는 “왜 이 변경을 넣었는가”와 “당시 어떤 위험이 지적됐는가”를 보완하는 1차 보조 근거로 사용한다.
5. 해석이 개입된 부분은 단정 대신 “보인다”, “해석된다”, “추정된다”라는 표현을 사용한다.

2 전체 연구 흐름의 상위 요약

Git 히스토리 전체를 크게 나누면 TA-BRPL 연구는 다음 여섯 단계로 전개되었다.

1. 2026년 1월 말: 최소 실험 플랫폼 구축과 selective forwarding 공격 프로토타입
2. 2026년 1월 말: 로그 기반 외부 trust engine 실험과 후처리 피드백 구조
3. 2026년 2월 초: Contiki-NG/BRPL 서브모듈 통합과 다중 고정 토폴로지 실험
4. 2026년 2월 초중반: trust sweep, fixed-topology 안정화, 결과 아카이빙
5. 2026년 3월 초: 병렬 실험 인프라와 재현 환경 정비
6. 2026년 3월 중후반: 논문화, 랜덤 토폴로지 일반화, endpoint 고정, ablation 확장

즉 이 연구는 처음부터 논문 중심으로 출발한 것이 아니라, “돌아가는 실험 시스템”을 먼저 만들고, 그 위에서 trust-aware routing을 실험하다가, 이후에 일반화와 학술적 주장 정리로 넘어간 형태다.

3 상세 연혁

3.1 2026-01-26: 최소 재현 가능한 실험 시스템 출발

대표 커밋 `de6fbbe` (2026-01-26 23:54:05 +0900), “Initial commit of trust-aware-brpl”

주요 추가 파일

- `brpl-of.c`
- `notes/receiver_root.c`, `notes/sender.c`
- `configs/simulation.csc`, `configs/simulation_ref.csc`
- `scripts/run_simulation.sh`, `scripts/build.sh`
- `tools/parse_results.py`
- `readme.md`

이 시점의 연구 상태 첫 커밋은 단순한 빈 저장소가 아니라, 이미 “한 번 돌려볼 수 있는” 형태를 갖춘 실험 플랫폼이었다. 초기 `readme.md`에는 다음 내용이 명시돼 있었다.

- 목표: BRPL에 trust 메커니즘을 붙여 selective forwarding 공격에 대응
- 1단계 구현 계획: 공격 모델, trust 계산, 방어 방식, 성능 평가
- 지표: PDR, end-to-end delay, overhead

해석 연구는 이론 정리보다 먼저 시뮬레이터 위에서 돌릴 수 있는 구조를 확보하는 방식으로 시작됐다. 첫날부터 root, sender, Cooja 설정, 결과 파서까지 포함돼 있다는 점은 이 프로젝트가 아이디어 메모가 아니라 “즉시 실험 가능한 프로토타입”으로 출발했음을 보여준다.

3.2 2026-01-27: 공격자 도입과 phase 기반 실험 초안

대표 커밋 26c504c (2026-01-27 03:12:15 +0900), “meow”

주요 변화

- `notes/attacker.c` 추가
- `configs/simulation_attack.csc`, `configs/simulation_normal.csc` 추가
- `notes/Makefile.attacker` 추가
- `memo.md`, `todo.md`, `tools/compare_scenarios.py` 추가
- `scripts/run_phase3.sh` 추가

이 시점의 의미 이 커밋에서 연구는 “평상시 라우팅”에서 “정상/공격 비교 실험” 단계로 넘어간다. 즉 공격자를 별도 mote로 분리하고, 시나리오를 normal과 attack으로 나누어, 실험 단위가 처음 정의된다.

해석 초기 연구 질문은 이 시점부터 더 구체화된다. 단순히 trust-aware routing을 만들자는 것이 아니라, 동일한 플랫폼에서 정상과 공격을 비교하고 차이를 지표로 뽑으려는 의도가 명확해진다.

3.3 2026-01-29 새벽: 랜덤 토폴로지와 내장 trust 개념의 등장

대표 커밋

- d293e43 (2026-01-29 02:32:48 +0900)
- 9995d2a (2026-01-29 05:11:02 +0900)
- 40f2446 (2026-01-29 05:20:57 +0900)

주요 변화

- random BRPL/MRHOF 시나리오용 `.csc` 템플릿 추가
- `scripts/gen_random_topology.py` 추가 및 수정

- docs/project_overview.md, docs/memo.md로 문서 정리 시작
- notes/brpl-trust.h 등장
- notes/brpl-blacklist.c/.h 축이 생성되기 시작
- tools/trust_engine/src/main.rs 초안 등장

당시 memo에 남은 trust 계산식 초기 메모에는 다음이 남아 있다.

$$\text{sample} = \frac{1000}{1 + \text{missed}}$$

즉, 노드별 마지막 sequence를 추적해 패킷 누락 수를 계산하고, 이를 EWMA trust 입력값으로 쓰는 단순 모델이 처음부터 의도돼 있었다.

해석 이 구간은 굉장히 중요하다. 연구는 아직 fixed topology 최적화에 들어가기 전 인데도, 이미 random topology, trust table, blacklist, 외부 trust engine 초안이 동시에 나타난다. 이는 연구자가 초기에 여러 가능성을 병렬 탐색했다는 뜻이다.

3.4 2026-01-29 저녁: 외부 trust engine과 로그 기반 피드백 루프 구축

대표 커밋 21eabf1 (2026-01-29 18:53:59 +0900), “add-trust-engine”

주요 추가 파일

- docs/external_trust_engine.md
- docs/jvm_crash_workaround.md
- scripts/run_with_trust_engine.sh
- scripts/run_sim_with_trust_post.sh
- tools/trust_engine/*

무엇이 달라졌나 이 시점부터 trust 계산은 두 경로를 가진다.

1. 시뮬레이터 내부에서 직접 계산하는 내장형 trust
2. Cooja 로그를 밖으로 빼서 Rust 엔진이 계산하고 다시 주입하는 외부형 trust

연구사적 의미 이 커밋은 “trust 메커니즘의 정답을 이미 알고 있었다”기보다, “일단 관측 가능한 모든 경로를 열어두고 실험해 보자”는 태도를 보여준다. 특히 trust_updates.txt

를 통해 후처리 결과를 다시 mote에 넣는 구조는, 시뮬레이터 한계 때문에 구현을 포기하지 않고 우회하는 전형적인 연구 프로토타이핑 방식이다.

3.5 2026-01-29 밤에서 2026-01-30 새벽: 모니터링, quick test, 첫 보고서

대표 커밋

- 4780735 (2026-01-29 21:33:45 +0900)
- a45b357 (2026-01-29 22:39:36 +0900)
- fc57c6e (2026-01-30 05:54:02 +0900)
- 12496c0 (2026-01-30 05:55:26 +0900)
- 8163291 (2026-01-30 06:02:53 +0900)

추가된 실험 지원 요소

- scripts/monitor_experiments.sh, watch_progress.sh
- scripts/quick_test.sh, quick_verify.sh
- scripts/run_experiments.sh
- docs/report/report_kr.tex, PDF 및 그림 자산

해석 이 시점은 알고리즘 자체보다, “실험이 돌아가는지 빨리 확인하는 방법”과 “결과를 바로 문서로 남기는 루프”가 만들어진 시기다. 첫 한글 보고서가 이때 이미 나온다는 점은, 연구자가 실험 결과를 짧은 주기로 해석하고 정리했음을 뜻한다.

3.6 2026-02-06: Contiki-NG/BRPL 서브모듈 통합

대표 커밋

- 84cf503 (2026-02-06 17:10:05 +0900), “add submodule brpl”
- bf9f511 (2026-02-06 17:27:37 +0900), “Refresh trust model and reporting”
- dc12219 (2026-02-06 21:00:22 +0900), “Integrate contiki-ng-brpl submodule and add trust-aware features”

주요 변화

- contiki-ng-brpl 서브모듈 연결

- `notes/brpl-trust.c` 추가
- `docs/architecture.md` 추가
- `configs/topologies/T1_S/M/L, T2_S/M/L, T3` 추가
- `scripts/gen_topology.py` 추가

연구 구조상의 전환 이전까지는 실험 저장소 주변부에서 `trust`를 붙였다 떼는 탐색이 많았다면, 이 시점부터는 Contiki/BRPL 기반 본체와 결합된 형태로 연구가 재정렬된다. 즉 “외부 실험”에서 “프로토콜 내부 메커니즘”으로 중심축이 이동한다.

해석 Git 상으로 보면 이 날은 연구 초반의 가장 큰 구조 전환점이다. 다중 고정 토폴로지와 `trust-aware` feature가 한 번에 들어오면서, 이후의 모든 시행착오가 동일한 시스템 틀 안에서 축적될 수 있게 된다.

3.7 2026-02-07: trust sweep 자동화와 안정화된 파이프라인

대표 커밋

- `fd3d6ec` (2026-02-07 02:54:46 +0900), “temp trust sim”
- `ef82dc5` (2026-02-07 03:14:17 +0900), “Add trust sweep automation and reporting”
- `5036811` (2026-02-07 08:21:48 +0900), “stable pipeline”
- `84379ce` (2026-02-07 21:46:58 +0900), “stabled-ver-grayhole”

주요 변화

- `scripts/run_trust_sweep.py` 추가
- `scripts/experiment_summary.py` 추가
- 대량 실험 결과를 `archive/` 아래 보관하는 구조 정착
- grayhole 계열 안정화 버전 등장

무엇이 연구적으로 중요했나 이때부터 연구는 한두 개 시나리오를 손으로 돌리는 수준을 넘어서, 파라미터 조합을 체계적으로 훑고 결과를 보관하는 단계로 올라간다. 나중에 `v1--v13.12` 시행착오 문서가 가능해진 것도, 실험이 이 시점부터 아카이브 형태로 누적됐기 때문이다.

3.8 2026-02-08에서 2026-02-10: sinkhole 안정화와 보고서 누적

대표 커밋

- 25c44b0 (2026-02-08 02:39:05 +0900), “stabled sinkhole”
- 9775283 (2026-02-08 17:59:32 +0900), “experiment stabled”
- e09bd81 (2026-02-09 20:46:41 +0900), “stabled ver maybe.”
- bb40900 (2026-02-09 21:47:00 +0900), “poll to temp var, optimize sim”
- 0e8d769 (2026-02-10 01:42:55 +0900), “add report”
- 31eb9e0 (2026-02-10 05:45:49 +0900), “add kr report”
- 40152f7 (2026-02-10 06:31:49 +0900), “update fig”

이 시기의 특징

- 공격자 코드, sender 코드, trust engine, 실행 스크립트가 계속 같이 수정됨
- 실험 안정화와 성능 최적화가 병행됨
- 영어/한글 보고서가 축적됨

해석 이 시기는 어떤 단일 아이디어를 증명한 시점이라기보다, trust-aware routing을 실제 sinkhole 실험에서 “덜 깨지게” 만드는 과정으로 보는 것이 맞다. 즉 논문 메시지 이전에 시스템의 생존성을 확보하는 단계였다.

3.9 2026-02-11: 회귀 테스트 도입

대표 커밋 6545a3d (2026-02-11 08:06:59 +0900),
“test: add trust engine regression checks for blacklist and exposure metrics”

의미 이 커밋은 파일 수로는 크지 않지만, 연구 방법론적으로는 중요하다. 이전까지의 실험이 “좋아 보이는 결과 찾기”에 가까웠다면, 이 시점부터는 blacklist와 exposure metric에 대해 기존 동작이 깨지지 않는지 확인하는 회귀 관점이 들어온다.

해석 수정이 누적될수록 우연히 맞춘 동작이 깨질 위험이 커지기 때문에, 연구도 자연스럽게 테스트 가능성을 요구하기 시작한 것으로 해석된다.

3.10 2026-02-23에서 2026-03-04: 환경 재현성과 실험 운영성 정비

대표 커밋

- 219501e (2026-02-23 14:09:09 +0900), “add nix set”
- ec3a2ea (2026-02-23 16:27:04 +0900), “add .gitattributes”
- b762f17 (2026-02-24 09:14:30 +0900), “update report sections and add LaTeX to nix environment”
- 628763e (2026-03-04 18:48:15 +0900), 연구용 PC 환경 셋업 스크립트 추가
- 9a2011e (2026-03-04 18:58:50 +0900), Java 21 업그레이드
- 67ddf37 (2026-03-04 19:00:56 +0900), 재시작 시 임시 파일 및 프로세스 정리
- c0abbf9 (2026-03-04 19:20:05 +0900), 빌드 캐시 재사용 제거

해석 이 구간은 알고리즘 변화보다도 “실험을 안정적으로 많이 돌릴 수 있는 환경”을 만드는 데 초점이 있다. 연구가 점점 대형 sweep과 논문화로 옮겨가면서, 재현성 문제와 환경 변수 문제를 먼저 치워야 했던 것으로 보인다.

3.11 2026-03-05: 병렬 실험 인프라와 논문 초안의 시작

대표 커밋

- 7bce50c (2026-03-05 14:08:29 +0900), “add parallel experiment”
- 99ada2d (2026-03-05 14:12:59 +0900), 병렬실험스크립트추가
- fb4bf36 (2026-03-05 16:12:36 +0900), 실험 안정화 및 파라미터/구조 수정
- 6446be9 (2026-03-05 17:34:49 +0900), 파라미터/구조 변경 반영
- 9992e67 (2026-03-05 20:11:14 +0900), “stable ver”
- aeb0499 (2026-03-05 20:52:43 +0900), “start writing paper”

주요 변화

- `.parallel_worker.env/` 구조 도입
- worker별 독립 build/runtime 사본 운영
- `scripts/run_experiments_parallel.sh`, `run_experiments_worker.sh`, `monitor_parallel.sh` 추가
- 같은 날 밤 `docs/paper/paper_draft.tex`와 `references.bib` 생성

연구사적 의미 이 날은 두 가지가 동시에 일어난다.

1. 실험 인프라가 소규모 배치 수준에서 병렬 대량실험 수준으로 확장된다.

2. 결과를 논문 문장으로 다시 써보는 단계가 시작된다.

즉, 연구는 이 시점부터 “잘 되는지 찾는 단계”에서 “무엇을 주장할 수 있는지 정리하는 단계”로 넘어간다.

3.12 2026-03-16에서 2026-03-19: 워크스페이스 정리와 문서 버전업

대표 커밋

- 859ca18 (2026-03-16 11:57:17 +0900), workspace snapshot sync
- 8bbc520 (2026-03-16 12:00:11 +0900), 임시/대용량 산출물 ignore
- 40f9150 (2026-03-19 05:57:09 +0900), “docs v0.3.3”

해석 이 구간은 폭발적으로 커진 실험 산출물을 정리하고, 문서의 기준 버전을 잡는 단계로 보인다. 3월 24일의 대규모 문서 통합을 앞둔 정리 국면에 가깝다.

3.13 2026-03-24 새벽: 통합 문서화와 메커니즘 재정리

대표 커밋 b18f19d (2026-03-24 04:33:38 +0900),
“feat: integrate TA-BRPL trust updates, unified docs, and refreshed figures”

주요 변화

- docs/TA_BRPL_UNIFIED.md 추가
- 기존의 ARCHITECTURE.md, IMPLEMENTATION_PLAN.md, RISKS_AND_LIMITATIONS.md, VALIDATION_PLAN.md 삭제
- docs/paper/paper.tex 본문 생성
- trust 관련 Makefile 조합과 figure 자산 다수 추가
- motes/ta-brpl-trust.c, project-conf.h 대규모 수정

해석 이 커밋은 코드와 문서의 동시 재구성이다. 흩어져 있던 설계 설명을 하나의 통합 문서로 모으고, 논문 본문도 새로 시작한다. 이는 연구자가 이제 TA-BRPL을 “이러저러한 실험 집합”이 아니라 “설명 가능한 메커니즘”으로 정리하기 시작했음을 뜻한다.

3.14 2026-03-24 오전: 랜덤 토폴로지 프레임워크의 본격 도입

대표 커밋

- 03106a5 (2026-03-24 05:42:23 +0900),
“feat: add random-topology framework and full paper pseudocode updates”
- ba86e46 (2026-03-24 06:23:55 +0900),
“Add sweep automation, loss/attack scenarios, and README run matrix”
- 74a05c4 (2026-03-24 22:48:56 +0900),
“Add robust sweep orchestration, rich monitoring, storage controls, and expanded random topology sweep”

주요 변화

- configs/scenarios/random_topo/manifest.json 추가
- sparse/medium/dense density별 시나리오 생성
- RPL, BRPL, SMTRUST, TABRPL 비교군 확장
- scripts/generate_random_topologies.py
- scripts/run_random_topo_sweep.sh
- sweep monitoring, storage control, all-sweeps orchestration 스크립트 추가

연구 질문의 변화 이 시점에서 연구의 중심 질문은 명백히 바뀐다. 더 이상 “고정 topology에서 잘 되는가”가 아니라, “topology distribution 전체에서 attacker isolation 방향성이 유지되는가”가 핵심 검증 질문이 된다.

관련 문서의 증거 docs/random_topo.md는 이 전환을 거의 선언문처럼 적고 있다. 핵심 문장은 다음과 같다.

고정 grid 한 장으로 승부 보는 설계는 버리고, 통제된 랜덤 토폴로지 분포 위에서 반복 평가하는 방식으로 가야 한다.

해석 이 문장은 연구가 fixed-topology optimization에서 generalization-oriented evaluation으로 넘어갔음을 가장 직접적으로 보여준다.

3.15 2026-03-27에서 2026-03-28: endpoint 고정과 실패의 체계적 문서화

대표 문서 및 커밋

- docs/MAIN_EXPERIMENT_PROTOCOL.md (작성일 2026-03-28)
- docs/base.md (작성일 2026-03-28)

- docs/TA_BRPL_V1_TO_NOW_POSTMORTEM.md (작성일 2026-03-29)
- 18c0dd7 (2026-03-28 05:50:53 +0900), “stabled ver”

무엇이 확정되었나

- 메인 메시지는 PDR 절대 우위가 아니라 isolation-with-bounded-cost로 고정
- primary endpoint는 att_share, hit_ratio
- secondary endpoint는 churn
- tertiary endpoint는 PDR_dur

이 시기의 연구적 성숙 가장 중요한 점은 연구자가 주장을 줄였다는 것이다. 즉 “모든 지표에서 다 좋아졌다”가 아니라, “공격자 의존 감소를 1차 목표로 두고, churn 비용을 상한 내에서 관리한다”는 더 방어 가능한 메시지를 선택했다.

postmortem의 의미 TA_BRPL_V1_TO_NOW_POSTMORTEM.md는 단순 회고가 아니라, 왜 v1에서 churn이 폭증했고, 왜 v8에서 T_fwd attribution bias가 문제가 되었으며, 왜 v13.8과 v13.12가 전환점이 되었는지를 정량값과 함께 정리한다.

3.16 2026-03-30: ablation 확장과 random-topology 보완 실험

대표 커밋

- 5cd74bf (2026-03-30 01:29:59 +0900)
- a55c3c1 (2026-03-30 06:08:19 +0900)

주요 변화

- random worker 환경에서 실패 상태 산출물이 대거 축적
- TABRPL_FWD, TABRPL_FWDCTRL 시나리오 추가
- rt_check용 축약 검증 세트 추가
- 한국어 논문 파일 docs/paper/main_ko.tex와 관련 결과 자산 갱신

해석 이 시점의 관심사는 전체 모델뿐 아니라 “어떤 신뢰 축이 실제 개선을 만드는가”를 분해하는 데 있다. 즉 연구는 단순 성능 비교를 넘어서, 메커니즘 기여도 분석 (ablation) 단계에 진입한 것으로 읽힌다.

4 TA-BRPL 모델링 진화의 상세 복원

4.1 초기 모델: seq-miss 기반 trust와 외부 주입형 구조

1월 말 메모와 초기 trust engine 흔적을 보면, 출발점은 비교적 단순했다. root가 각 송신자의 마지막 sequence를 추적하고, 누락 패킷 수를 기반으로

$$\text{sample} = \frac{1000}{1 + \text{missed}}$$

형태의 score를 만든 뒤 EWMA로 누적하는 구조였다. 이 시기의 핵심은 “정교한 trust 정의”보다 “관측 가능한 손실을 trust 신호로 바꿀 수 있는가”였다.

중요한 점은 이 trust가 처음부터 완성된 in-protocol 메커니즘은 아니었다는 것이다. 1월 29일 전후의 외부 trust engine 실험은 Cooja 로그를 Rust 엔진으로 보내고, 후처리 결과를 다시 mote에 주입하는 우회 경로를 열어 둔다. 이는 연구 초반의 관심이 엄밀한 모델 완성보다, 일단 trust를 계산하고 실험 루프를 닫는 데 있었음을 보여준다.

4.2 2026-02-06 전환: forwarding-ratio와 exposure 중심으로 재정의

2월 6일의 bf9f511과 같은 날의 GitHub PR zeetee1235/TA-BRPL#1은 첫 번째 큰 모델 전환을 보여준다. PR 제목 자체가 Refresh trust model: EWMA from forwarding ratios, exposure reporting, and docs였다.

이 전환의 핵심

- trust 입력을 단순 seq-miss 해석에서 forwarding ratio 기반으로 바꿈
- 공격 노출 정도를 표현하는 exposure metric을 함께 기록함
- threshold 해석을 단순 정수 score가 아니라 확률적 trust scale에 가깝게 재정리함
- parser와 summary/reporting까지 함께 수정해 실험 해석 축 자체를 바꿈

왜 중요했는가 이 전환은 단순 수치 조정이 아니라 “무엇을 악의 증거로 볼 것인가”의 변경이다. seq-miss는 손실 현상만 포착하지만, forwarding ratio는 “이 parent가 받은 패킷을 실제로 얼마나 넘겼는가”에 더 직접적으로 연결된다. 즉 TA-BRPL trust는 이 시점부터 네트워크 결과 관측에서 forwarding 행위 관측으로 조금 더 메커니즘 진화적으로 이동한다.

GitHub 리뷰가 보여준 한계 같은 PR의 리뷰에서는 `udp_to_root` 증가가 없더라도 `dropped` 값은 늘 수 있는데, 이 경우 `trust` 업데이트가 건너뛰어질 수 있다는 지적이 남아 있다. 이는 초창기 모델이 “전달 성공”에 지나치게 기대고 있어서, 실패 증거가 특정 조건 아래에서 누락될 수 있었음을 보여준다.

4.3 초기 in-protocol trust adapter의 의미

2월 6일에 추가된 `motest/brpl-trust.c`는 후대의 완성형 TA-BRPL 모델과는 성격이 다르다. 이 파일은 본질적으로

- 노드별 `trust` value 저장
- `trust` validity 표시
- `parent` 허용/차단 판정
- 외부 또는 상위 로직의 `trust` override 반영

를 담당하는 얇은 adapter에 가깝다.

즉 이 단계에서는 `trust`의 의미를 프로토콜 내부에서 풍부하게 계산한다기보다, “이미 계산된 `trust`를 BRPL `parent` 선택에 연결하는 접착층”이 먼저 만들어졌다. 이 점은 연구가 모델 정의와 라우팅 결합을 동시에 완성한 것이 아니라, 결합 지점을 먼저 확보한 뒤 모델을 뒤늦게 풍부하게 만든 흐름이었음을 말해 준다.

4.4 2026-02-07 전후: `lambda/gamma` sweep과 파라미터화

GitHub PR `zeetee1235/TA-BRPL#2 Add trust sweep runner, summary/reporting scripts, and expose TRUST_LAMBDA/TRUST_GAMMA`는 모델이 단일 heuristic에서 실험 가능한 parameterized model로 이동했음을 보여준다.

이 시점의 변화

- `TRUST_LAMBDA`를 외부 노출해 EWMA 민감도를 조절 가능하게 함
- `TRUST_GAMMA`를 외부 노출해 `trust` penalty 강도를 바꿀 수 있게 함
- `sweep runner`와 `summary` 스크립트로 파라미터 탐색을 자동화함

이 변화의 의미는 명확하다. 연구자는 더 이상 “좋아 보이는 `trust` 상수 하나”를 찾으려 하지 않고, 신뢰 업데이트 속도와 `routing` penalty의 민감도를 분리해 탐색하기 시작했다. 즉 모델이 수식 하나의 문제가 아니라, 동적 반응성과 `parent` 전환 비용 사이의 trade-off 문제라는 인식이 이 시점에 자리 잡는다.

흥미롭게도 이 PR의 리뷰에서는 `topology` 이름에 `underscore`가 들어갈 때 `parser` regex

가 깨질 수 있다는 지적도 나온다. 이는 모델 자체뿐 아니라 실험 자동화와 결과 해석 도구가 함께 안정화되어야만 연구가 축적될 수 있었음을 보여준다.

4.5 2026-03 중반 이후: on-node 3축 trust 모델의 형성

3월 중반 이후 `motest/ta-brpl-trust.c`가 본격적으로 커지면서, TA-BRPL은 단순 `trust scalar`를 넘어서 다중 증거 결합 모델로 발전한다. 특히 3월 24일의 `b18f19d`는 이 전환을 문서와 코드 양쪽에서 동시에 보여준다.

핵심 신호 축

- `T_fwd`: forwarding behavior에 대한 신뢰
- `T_ctrl`: control-plane anomaly에 대한 신뢰
- `T_hon`: queue honesty 또는 self-report 계열의 정직성 신호

이 신호들은 가중 기하평균과 비대칭 EWMA 같은 형태로 결합되며, 단순한 “평균 trust”보다 공격 유형별 증거를 더 세밀하게 분리하려는 방향을 가진다. 연구의 초점도 여기서 달라진다. 이제 질문은 “trust가 높은가”가 아니라 “어떤 종류의 악성 행동이 어떤 축에서 먼저 드러나는가”가 된다.

4.6 `trust_fwd` 분리와 blackhole dilution 문제

이 문서 개정에서 가장 중요하게 보강된 지점은 `trust_fwd` 분리의 의미다. 3월 24일 전후의 코드 주석과 구조는, aggregate trust 하나만으로는 blackhole evidence가 희석될 수 있다는 문제를 아주 분명하게 드러낸다.

왜 희석이 발생하는가

- 어떤 노드가 forwarding은 악의적으로 떨어뜨려도
- control-plane 신호나 honesty 신호는 겉으로 양호할 수 있고
- 이 경우 전체 결합 trust는 생각보다 높게 남을 수 있다

즉 blackhole류 공격에서는 forwarding 관련 증거가 가장 직접적인데, 이를 총합 trust 안에만 넣으면 다른 축이 이를 덮어버릴 수 있다. 그래서 TA-BRPL은 최종적으로 forwarding evidence를 따로 보는 `trust_fwd`를 독립적으로 추적하게 된다.

이는 모델이 더 복잡해졌다는 뜻만이 아니다. 공격 탐지에서 “soft trust aggregation”과 “attack-specific veto signal”을 분리하는 방향으로 철학이 바뀌었다는 뜻이다.

4.7 validation layer의 등장: soft trust와 hard exclusion의 분리

3월 후반 모델은 trust score만으로 바로 parent를 끊지 않는다. review, validation, release, blacklist 상태가 추가되면서, soft trust 저하와 hard exclusion이 별도의 층으로 분리된다.

이 분리가 중요한 이유

- trust는 noisy할 수 있으므로 바로 blacklist하면 churn이 폭증할 수 있다
- 반대로 명백한 forwarding 악성 증거는 review/validation을 거쳐 빠르게 격리해야 한다
- 따라서 “관찰”, “검토”, “격리”, “해제”가 단계적으로 구성된다

이는 2월의 단순 threshold형 trust에서 3월 말의 validation-aware trust control로 연구가 진화했음을 보여준다. 즉 TA-BRPL의 최종 형태는 trust metric 그 자체보다, trust를 routing action으로 바꾸는 상태기계에 더 가깝다.

4.8 BRPL 결합 함수의 진화: penalty에서 hysteresis-aware gating으로

서브모듈 contiki-ng-brpl의 `os/net/routing/rpl-classic/rpl-brpl.c` 이력은, TA-BRPL의 모델링이 로컬 저장소 안에서만 진화한 것이 아님을 보여준다.

주요 전환 커밋

- 26bb815: trust-aware routing modification 초기 결합
- 559e51b: sinkhole/grayhole trust 반영 확대
- 094b502: routing behavior 조정
- 521724e: parent selection hook과 hysteresis gating 정교화

초기 결합은 trust 값을 BRPL weight에 직접 penalty로 먹이는 접근에 가까웠다. 그러나 시간이 갈수록 parent switch margin, dwell time, preferred-parent 추적, validation penalty scale, hard exclude hook이 들어오면서 구조가 바뀐다.

이 흐름이 의미하는 것

- 낮은 trust를 단순히 metric에 곱하는 것만으로는 안정적 행동이 나오지 않음
- routing은 hysteresis와 re-selection 경로까지 포함해 제어해야 함

- 공격자 격리와 churn 억제는 trust score 하나로 해결되지 않고, parent switching policy 전체를 함께 조정해야 함

GitHub PR zeetee1235/contiki-ng-brpl#2와 #3의 리뷰도 이 점을 뒷받침한다. 한 리뷰는 낮은 trust가 오히려 더 좋은 score로 해석될 수 있는 방향성 버그를, 다른 리뷰는 전역 dwell tracking이나 fallback re-selection 때문에 배제한 parent가 다시 선택될 위험을 지적한다. 이는 TA-BRPL 모델링의 핵심 난제가 “trust를 어떻게 계산할까”만이 아니라, “그 trust를 BRPL 행동으로 옮길 때 어떤 역효과가 생기는가”였음을 잘 보여준다.

4.9 정리: TA-BRPL 모델링 철학은 어떻게 바뀌었는가

Git, GitHub, 서브모듈 이력을 합쳐 보면 TA-BRPL 모델링은 다음 순서로 정교화되었다.

1. 초기에는 loss 기반 seq-miss heuristic과 외부 trust 주입 실험이 있었다.
2. 2월 초에는 forwarding ratio와 exposure 중심으로 trust 정의를 재정의했다.
3. 곧바로 TRUST_LAMBDA, TRUST_GAMMA를 분리 노출해 동적 반응성과 penalty 민감도를 탐색했다.
4. 3월에는 T_fwd, T_ctrl, T_hon의 3축 모델로 확장했다.
5. blackhole dilution 문제를 인식하면서 trust_fwd를 독립 추적했다.
6. validation/review/release 계층을 넣어 soft trust와 hard exclusion을 분리했다.
7. 마지막으로 BRPL 내부의 hysteresis, gating, parent fallback까지 조정하며 “trust score”가 아니라 “validation-aware routing control”에 가까운 형태로 수렴했다.

따라서 TA-BRPL의 연구사는 단순히 trust 수식을 개선한 과정이 아니다. 오히려 관측 신호 선택, 공격 유형별 증거 분리, blacklist 남용 방지, parent switch 안정화, 그리고 공격자 격리와 churn 비용 사이의 균형점을 찾아가는 과정이었다고 정리하는 편이 더 정확하다.

5 요약 타임라인 표

날짜	대표 커밋	기술적 변화	연구 의미
2026-01-26	de6fbbe	root/sender, Cooja, 초기 parser, README	최소 실험 시스템 출발
2026-01-27	26c504c	attacker, normal/attack 시나리오	공격 비교 실험 시작
2026-01-29	9995d2a 외 새벽	random topology, brpl-trust, docs 정리	trust 개념과 실험 축 확장
2026-01-29	21eabf1	Rust trust engine, post-processing injection	외부 trust 실험 루프 구축 저녁
2026-01-30	12496c0 외	monitor, quick test, 첫 보고서	실험 운영 루프 형성
2026-02-06	dc12219	Contiki submodule, 다중 topology, trust file 통합	본격적인 TA-BRPL 시스템화
2026-02-07	ef82dc5, 5036811	trust sweep, archive, stable pipeline	대량 탐색과 결과 축적 시작
2026-02-08-10	25c44b0 외	sinkhole 안정화, 최적화, 보고서 누적	fixed-topology 동작 안정화
2026-02-11	6545a3d	regression checks	누적 실험의 검증 체계 도입
2026-02-23-03-04	219501e 외	Nix, Java 21, cleanup, setup scripts	환경 재현성 및 운영성 정비
2026-03-05	7bce50c, aeb0499	병렬 실험, paper draft	대량실험 + 논문화 동시 시작
2026-03-24	b18f19d	unified docs, trust updates, paper 재작성	메커니즘 설명 체계 정리
2026-03-24	03106a5, 74a05c4	random-topology framework, orchestration	일반화 검증으로 연구 초점 이동
2026-03-28	18c0dd7 + 문서군	endpoint 고정, post-mortem 축적	주장 범위와 성공 기준 고정

날짜	대표 커밋	기술적 변화	연구 의미
2026-03-30	5cd74bf, a55c3c1	FWD/FWDCTRL ablation, rt.check	기여도 분해와 보완 실험 단계

6 복원으로부터 읽히는 연구 방향의 변화

6.1 1단계: “일단 돌려보자”

초기 연구는 selective forwarding 공격과 trust-aware routing을 Cooja에서 어떻게든 재현하는 데 집중돼 있었다. 구현의 완결성보다 실험 가능성이 우선이었다.

6.2 2단계: “trust를 어디에 넣어야 하는가”

1월 말에서 2월 초 사이에는 trust를 root에서 계산할지, 외부 엔진으로 계산할지, BRPL 점수에 어떻게 연결할지, blacklist를 얼마나 강하게 둘지 등 설계 공간을 넓게 탐색했다.

6.3 3단계: “고정 토폴로지에서 왜 망하는가”

2월의 긴 구간은 파라미터 스윙과 안정화 시도였다. 나중의 postmortem이 보여주듯, 이 시기의 핵심 문제는 “trust를 넣으면 좋아질 것”이라는 직관이 틀렸다는 사실을 받아들이고, churn 폭증과 attribution bias를 구조 문제로 진단한 데 있다.

6.4 4단계: “일반화되는가”

3월 중후반부터는 연구 질문 자체가 바뀐다. 고정 topology에서 좋게 보인 결과가 random topology 분포에서 유지되는지, 그리고 그 결과를 어떤 endpoint로 주장할 수 있는지가 핵심이 된다.

6.5 5단계: “무엇을 성공이라고 부를 것인가”

메인 protocol 문서가 보여주듯, 최종적으로는 PDR 절대 우위를 버리고 attacker isolation과 bounded churn cost를 핵심 메시지로 택하게 된다. 이는 결과가 나빠서 후퇴했다기보다, 관찰된 trade-off를 정직하게 반영한 정제된 주장으로 보는 편이 맞다.

7 최종 복원 결론

Git 히스토리 기준으로 TA-BRPL 연구는 다음과 같이 복원된다.

1. 처음에는 selective forwarding 대응용 BRPL 실험 플랫폼을 빠르게 만들었다.
2. 곧바로 공격자, trust 계산, 랜덤 토폴로지, 외부 trust engine을 병렬 탐색했다.
3. 2월 초에 Contiki-NG/BRPL 본체와 통합하면서 시스템 연구 형태로 전환했다.
4. 이후 고정 토폴로지에서 trust 정책의 실패와 churn 문제를 반복적으로 진단했다.
5. 3월 초에는 병렬 인프라와 환경 정비로 대량실험 기반을 만들었다.
6. 3월 중후반에는 random topology 일반화와 논문화가 연구의 중심이 되었다.
7. 최종적으로는 “PDR 향상 알고리즘”이 아니라 “공격자 의존 감소를 우선하고 churn 비용을 관리하는 trust-aware BRPL 설계”로 문제 정의가 정리되었다.

따라서 이 저장소의 연구사는 단순한 코드 추가 연속이 아니라, **프로토타입 구축 → trust 실험 확장 → 시스템 통합 → 고정 토폴로지 튜닝 → 일반화 검증 → 논문화와 주장 정제**의 흐름으로 이해하는 것이 가장 자연스럽다.